

Deep Learning & PyTorch

Complete Reference Cheatsheet

From Fundamentals to Advanced Practice

PyTorch · Deep Learning · Transformers · Large Language Models · Reinforcement Learning

-
-  **Part I–III** PyTorch: Basic → Advanced
 -  **Part IV** Deep Learning Foundations
 -  **Part V** Transformers
 -  **Part VI** Large Language Models: Basic → Advanced
 -  **Part VII** Reinforcement Learning
 -  **Appendix** Errors, Shapes, Glossary, Index

Contents

I	PyTorch — Fundamentals	4
1	Tensors	4
1.1	Creating Tensors	4
1.2	Attributes, Dtype & Device	4
1.3	Indexing, Slicing & Reshaping	4
1.4	Operations & Broadcasting	5
2	Autograd (Automatic Differentiation)	5
3	nn.Module & Layers	5
3.1	Common Building-Block Layers	6
3.2	Activation Functions (Quick Reference)	6
4	Loss Functions	6
5	Optimizers	6
6	Data Loading: Dataset & DataLoader	7
7	The Basic Training Loop	7
II	PyTorch — Intermediate	8
8	Devices, GPUs & CUDA	8
8.1	Simple Multi-GPU (DataParallel vs DDP)	8
9	Learning Rate Schedulers	8
10	Saving & Loading Models	8
11	Regularization Layers in Practice	9
12	Convolutional Networks (CNNs)	9
13	Recurrent Networks (RNN / LSTM / GRU)	9
14	Custom Transforms & torchvision	10
15	Evaluation & Metrics	10
16	Logging: TensorBoard	10
III	PyTorch — Advanced	10
17	Custom Autograd Functions	10
18	Hooks	11
19	Mixed Precision Training (AMP)	11
20	Distributed Training (DDP)	11
21	Compilation & Export	12
22	Quantization	12
23	Profiling & Performance Debugging	12
24	Gradient Checkpointing (Activation Recomputation)	13

IV Deep Learning — Foundations	13
25 The Neuron & Backpropagation	13
26 Weight Initialization	13
27 Normalization Techniques	13
28 Optimization Algorithms	14
29 Regularization & Generalization	14
30 Vanishing & Exploding Gradients	14
31 CNN Architecture Evolution (Quick Reference)	14
32 Hyperparameter Tuning Checklist	14
V Transformers	15
33 Why Transformers? (Motivation)	15
34 Self-Attention	15
35 Multi-Head Attention	15
36 Positional Encoding	16
37 The Transformer Block	16
38 Encoder–Decoder Architecture	16
39 Vision Transformer (ViT)	17
40 Efficient & Long-Context Attention	17
VI Large Language Models (LLMs)	17
41 Tokenization	17
42 Embeddings & Input Representation	18
43 Pretraining Objectives	18
44 Model Family Tree	18
45 Fine-Tuning: Full vs. Parameter-Efficient (PEFT)	18
46 LoRA (Low-Rank Adaptation)	18
47 Instruction Tuning, RLHF & DPO	19
48 Decoding / Generation Strategies	19
49 KV-Cache & Inference Optimization	19
50 Quantization for LLMs	20
51 Extending Context Length	20
52 Prompt Engineering Quick Reference	20
53 Evaluation Metrics	20
54 Hugging Face transformers: Quick Start	20

55 Scaling Laws (Brief)	21
VII Reinforcement Learning	21
56 Markov Decision Processes (MDPs)	21
57 Bellman Equations	21
58 Value-Based Methods: Q-Learning & DQN	22
59 Exploration Strategies	22
60 Policy Gradient Methods: REINFORCE	22
61 Actor-Critic & A2C	23
62 PPO (Proximal Policy Optimization)	23
63 RL for LLMs: RLHF Recap	23
64 Quick Reference: Algorithm Families	24
VIII Appendix	24
65 Common PyTorch Errors & Fixes	24
66 Shape Debugging Cheatsheet	24
67 Useful One-Liners	24
68 Glossary	25
69 Further Reading	25
Index	26

Part I PyTorch — Fundamentals

1 Tensors

A **Tensor** is PyTorch's core n-dimensional array, similar to a NumPy array but with GPU acceleration and autograd support.

1.1 Creating Tensors

```
import torch

torch.tensor([1, 2, 3])           # from a Python list
torch.zeros(2, 3)                 # 2x3 tensor of zeros
torch.ones(2, 3)                  # 2x3 tensor of ones
torch.full((2, 3), 7)             # filled with a constant
torch.eye(3)                      # identity matrix
torch.arange(0, 10, 2)            # [0, 2, 4, 6, 8]
torch.linspace(0, 1, steps=5)     # 5 evenly spaced points
torch.rand(2, 3)                  # uniform [0,1)
torch.randn(2, 3)                 # standard normal
torch.randint(0, 10, (2, 3))      # random ints
torch.empty(2, 3)                 # uninitialized memory
torch.from_numpy(np_array)        # from a NumPy array (shares memory)
torch.zeros_like(x); torch.ones_like(x) # match shape/dtype of x
```

1.2 Attributes, Dtype & Device

```
x = torch.randn(3, 4, dtype=torch.float32, device="cpu")
x.shape           # torch.Size([3, 4])
x.dtype           # torch.float32
x.device          # device(type='cpu')
x.ndim            # 2
x.numel()         # 12 (total number of elements)
x.requires_grad   # False by default

x = x.to(dtype=torch.float16)      # cast dtype
x = x.to("cuda")                  # move to GPU (if available)
x = x.cuda(); x = x.cpu()          # shortcuts
x.type()                       # full type string
```

Common dtypes: `torch.float32` (default, float), `torch.float16` / `torch.bfloat16` (mixed precision), `torch.float64` (double), `torch.int64` (long, used for labels/indices), `torch.bool`.

1.3 Indexing, Slicing & Reshaping

```
x = torch.arange(24).reshape(4, 6)

x[0]           # first row
x[:, 0]        # first column
x[1:3, 2:4]    # sub-block (rows 1-2, cols 2-3)
x[x > 10]      # boolean mask -> 1D tensor of matches
torch.where(x > 10, x, torch.zeros_like(x))

x.view(2, 12)   # reshape (requires contiguous memory)
x.reshape(2, 12) # reshape (copies if needed - safer than view)
x.flatten()     # 1D
x.unsqueeze(0)  # add dim at position 0 -> shape [1, 4, 6]
x.squeeze()     # remove dims of size 1
x.permute(1, 0) # reorder dims (like a generalized transpose)
x.transpose(0, 1) # swap two dims
x.T             # 2D transpose shortcut
x.contiguous()  # force contiguous memory layout
```

1.4 Operations & Broadcasting

```
a + b; a - b; a * b; a / b      # elementwise (broadcast if shapes compatible)
a @ b                          # matrix multiplication
torch.matmul(a, b)              # same as @
torch.bmm(a, b)                 # batched matmul, a: [B,n,m], b: [B,m,p]
a.sum(); a.sum(dim=0); a.mean(dim=1, keepdim=True)
a.max(dim=1); a.argmax(dim=1)
torch.cat([a, b], dim=0)        # concatenate along a dim
torch.stack([a, b], dim=0)       # new dim, stacks tensors
a.clamp(min=0, max=1)           # clip values
a.norm(p=2)                     # L2 norm
torch.einsum('bij,bjk->bik', a, b) # Einstein summation - flexible & fast
```

Broadcasting Rule

Two tensors are broadcastable if, aligning shapes from the *right*, every pair of dimensions is either equal, or one of them is 1 (or missing). Shape (3,1,5) broadcasts with (1,4,5) → (3,4,5).

2 Autograd (Automatic Differentiation)

Autograd builds a dynamic computation graph and computes gradients via reverse-mode differentiation (backpropagation).

```
x = torch.randn(3, requires_grad=True)
y = (x ** 2).sum()
y.backward()      # populates x.grad with dy/dx
print(x.grad)     # tensor([2x0, 2x1, 2x2])

with torch.no_grad():      # disable graph tracking (inference / manual updates)
    x -= 0.1 * x.grad

x.grad.zero_()            # reset gradients before next backward() call
x.detach()                # new tensor, same data, detached from graph
x.requires_grad_(True)    # in-place flag toggle
```

tensor.backward(gradient=None)

Computes gradients of a scalar (or pass a matching-shape **gradient** for a non-scalar) w.r.t. all leaf tensors with `requires_grad=True`.

torch.autograd.grad(outputs, inputs)

Returns gradients without accumulating into `.grad` – useful for higher-order gradients (e.g. meta-learning, WGAN-GP penalties).

tensor.retain_grad()

Keep `.grad` for a non-leaf tensor (normally freed after backward).

Common pitfall: gradients *accumulate* by default. Always call `optimizer.zero_grad()` (or `model.zero_grad()`) before each `backward()` unless you intentionally want gradient accumulation across steps.

3 nn.Module & Layers

Every model and layer subclasses **nn.Module**, which tracks parameters, submodules, and buffers automatically.

```
import torch.nn as nn
import torch.nn.functional as F

class MLP(nn.Module):
    def __init__(self, in_dim, hidden_dim, out_dim):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, out_dim)
        self.act = nn.ReLU()

    def forward(self, x):
        x = self.act(self.fc1(x))
        return self.fc2(x)

model = MLP(784, 256, 10)
```

```
out = model(torch.randn(32, 784))    # forward pass, calls __call__ -> forward()
```

3.1 Common Building-Block Layers

Layer	Purpose
<code>nn.Linear(in, out)</code>	Fully-connected affine layer: $y = xW^T + b$
<code>nn.Conv2d(cin, cout, k)</code>	2D convolution for images/feature maps
<code>nn.BatchNorm2d(c)</code>	Normalizes activations per-channel across the batch
<code>nn.LayerNorm(dim)</code>	Normalizes across the feature dimension (used in Transformers)
<code>nn.Dropout(p)</code>	Randomly zeroes units at rate p during training
<code>nn.Embedding(vocab, dim)</code>	Lookup table mapping integer IDs to dense vectors
<code>nn.LSTM</code> / <code>nn.GRU</code>	Recurrent layers for sequential data
<code>nn.MultiheadAttention</code>	Scaled dot-product multi-head attention (see §35)
<code>nn.Sequential(...)</code>	Chains layers/modules in order
<code>nn.ModuleList</code> / <code>nn.ModuleDict</code>	Containers that register submodules properly (unlike a plain Python list)

Use `nn.ModuleList`, never a raw Python list, to store submodules – otherwise their parameters won't be registered, moved by `.to(device)`, or found by `model.parameters()`.

3.2 Activation Functions (Quick Reference)

Function	PyTorch	Notes
ReLU	<code>nn.ReLU()</code>	$\max(0, x)$; fast, can "die" (zero gradient)
LeakyReLU	<code>nn.LeakyReLU(0.01)</code>	Small negative slope avoids dead units
GELU	<code>nn.GELU()</code>	Smooth; default in BERT/GPT/Transformers
SiLU / Swish	<code>nn.SiLU()</code>	$x \cdot \sigma(x)$; used in modern CNNs/LLMs
Sigmoid	<code>nn.Sigmoid()</code>	Squashes to $(0, 1)$; gates, binary output
Tanh	<code>nn.Tanh()</code>	Squashes to $(-1, 1)$; classic RNN gate
Softmax	<code>nn.Softmax(dim=-1)</code>	Converts logits to a probability distribution

4 Loss Functions

Loss	Use case
<code>nn.MSELoss()</code>	Regression; mean squared error
<code>nn.L1Loss()</code>	Regression; mean absolute error, robust to outliers
<code>nn.CrossEntropyLoss()</code>	Multi-class classification; expects raw logits + integer class labels (applies log-softmax internally)
<code>nn.NLLLoss()</code>	Use after your own <code>log_softmax</code>
<code>nn.BCELoss()</code>	Binary classification; expects probabilities (post-sigmoid)
<code>nn.BCEWithLogitsLoss()</code>	Binary classification; expects raw logits (numerically stable, preferred over BCELoss)
<code>nn.KLDivLoss()</code>	Distribution matching (e.g. distillation)
<code>nn.SmoothL1Loss()</code>	Huber-style loss; used in detection (bounding boxes)
<code>nn.CosineEmbeddingLoss()</code>	Similarity/embedding learning

```
criterion = nn.CrossEntropyLoss()
logits = model(x)          # shape [batch, num_classes], raw scores
loss = criterion(logits, targets) # targets: shape [batch], dtype long
```

5 Optimizers

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, betas=(0.9, 0.999))
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)

optimizer.zero_grad()    # clear old gradients
loss.backward()          # compute new gradients
optimizer.step()          # apply the update rule
```

Optimizer	Notes
SGD	Simple, needs tuning; + <code>momentum</code> to accelerate/reduce oscillation
Adam	Adaptive per-parameter LR (1st + 2nd moment estimates); strong default
AdamW	Adam with <i>decoupled</i> weight decay – preferred over Adam for Transformers/LLMs
RMSprop	Adaptive LR, popular in classic RL (e.g. DQN)
Adagrad	Adapts LR per-parameter based on historical gradient magnitude; good for sparse features

6 Data Loading: Dataset & DataLoader

```
from torch.utils.data import Dataset, DataLoader

class MyDataset(Dataset):
    def __init__(self, data, labels):
        self.data, self.labels = data, labels

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]

dataset = MyDataset(X, y)
loader = DataLoader(dataset, batch_size=32, shuffle=True,
                    num_workers=4, pin_memory=True, drop_last=False)

for batch_x, batch_y in loader:
    ...
```

`num_workers`

Parallel subprocesses for data loading (avoids CPU bottleneck).

`pin_memory=True`

Speeds up host→GPU transfer for CUDA training.

`collate_fn`

Custom function to merge a list of samples into a batch (needed for variable-length sequences, e.g. text).

7 The Basic Training Loop

Nearly every PyTorch project follows this same skeleton:

```
model = MLP(784, 256, 10).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()

for epoch in range(num_epochs):
    model.train()  # enable dropout / BN training mode
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)

        optimizer.zero_grad()
        logits = model(x)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()

    model.eval()  # disable dropout, freeze BN stats
    with torch.no_grad():
        for x, y in val_loader:
            x, y = x.to(device), y.to(device)
            val_loss = criterion(model(x), y)
```

Always call `model.train()` / `model.eval()` at the right times – Dropout and BatchNorm behave differently in each mode. Forgetting `eval()` is one of the most common silent bugs in PyTorch code.

Part II PyTorch — Intermediate

8 Devices, GPUs & CUDA

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
x = x.to(device)

torch.cuda.is_available()      # bool
torch.cuda.device_count()      # number of visible GPUs
torch.cuda.current_device()
torch.cuda.get_device_name(0)
torch.cuda.memory_allocated() / 1e9  # GB currently allocated
torch.cuda.empty_cache()          # release cached (unused) memory to the OS
torch.cuda.synchronize()         # block until all CUDA kernels finish (accurate timing)
```

Rule of thumb: model *and* data must be on the same device, or you'll get `RuntimeError: Expected all tensors to be on the same device`. Move the model once at setup; move each batch inside the loop.

8.1 Simple Multi-GPU (DataParallel vs DDP)

```
model = nn.DataParallel(model)      # quick, single-process, splits batch across GPUs
                                     # (legacy; DistributedDataParallel is preferred -- see Part III)
```

9 Learning Rate Schedulers

```
sched = torch.optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=50)
sched = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', patience=5)
sched = torch.optim.lr_scheduler.OneCycleLR(optimizer, max_lr=1e-3, total_steps=1000)
sched = torch.optim.lr_scheduler.LinearLR(optimizer, start_factor=0.1, total_iters=5)  # warmup

for epoch in range(num_epochs):
    train_one_epoch(...)
    sched.step()                    # most schedulers: call once per epoch
    # sched.step(val_loss)          # ReduceLROnPlateau needs the metric
```

Transformer / LLM training almost always pairs a short **linear warmup** with **cosine decay** – warmup avoids early instability from large adaptive-optimizer updates.

10 Saving & Loading Models

```
# Save (recommended: state_dict only, not the whole object)
torch.save(model.state_dict(), "model.pt")

# Load
model = MLP(784, 256, 10)
model.load_state_dict(torch.load("model.pt", map_location=device))
model.eval()

# Full checkpoint (resume training)
torch.save({
    "epoch": epoch,
    "model_state": model.state_dict(),
    "optimizer_state": optimizer.state_dict(),
    "loss": loss,
}, "checkpoint.pt")

ckpt = torch.load("checkpoint.pt", map_location=device)
model.load_state_dict(ckpt["model_state"])
optimizer.load_state_dict(ckpt["optimizer_state"])
```

Never `torch.save(model, ...)` (pickles the whole class/code) for shared checkpoints – it breaks if your class definition changes. Save/load `state_dict` instead.

11 Regularization Layers in Practice

```
self.drop = nn.Dropout(p=0.5) # zero units w.p. 0.5, scale rest by 1/(1-p)
self.bn = nn.BatchNorm2d(num_features=64) # normalize per-channel over the batch
self.ln = nn.LayerNorm(normalized_shape=512) # normalize over the feature dim (per-sample)

optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01) # L2-style penalty
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0) # gradient clipping
```

Technique	When to use
Dropout	Fully-connected layers, prone to overfitting on small data
BatchNorm	CNNs; needs reasonably large batch sizes to estimate stable statistics
LayerNorm	Transformers/RNNs; batch-size independent, works with sequence data
Weight decay	Almost always on for Adam-family optimizers (use AdamW's decoupled form)
Gradient clipping	RNNs, Transformers, RL – prevents exploding gradients from destabilizing training
Early stopping	Any setting where you track a validation metric
Data augmentation	Vision (flip, crop), text (back-translation), audio (noise/pitch)

12 Convolutional Networks (CNNs)

```
class SmallCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2), # 32x32 -> 16x16
            nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d(2), # 16x16 -> 8x8
            nn.AdaptiveAvgPool2d(1), # -> [B, 64, 1, 1]
        )
        self.classifier = nn.Linear(64, num_classes)

    def forward(self, x):
        x = self.features(x)
        x = x.flatten(1)
        return self.classifier(x)
```

Output size of a Conv/Pool layer

$$\text{out} = \left\lfloor \frac{\text{in} + 2 \cdot \text{padding} - \text{dilation} \cdot (\text{kernel} - 1) - 1}{\text{stride}} + 1 \right\rfloor$$

13 Recurrent Networks (RNN / LSTM / GRU)

```
lstm = nn.LSTM(input_size=100, hidden_size=256, num_layers=2,
               batch_first=True, bidirectional=False)

x = torch.randn(32, 50, 100) # [batch, seq_len, input_size]
out, (h_n, c_n) = lstm(x)
# out: [batch, seq_len, hidden_size] -- output at every timestep
# h_n: [num_layers, batch, hidden_size] -- final hidden state
# c_n: [num_layers, batch, hidden_size] -- final cell state (LSTM only)

gru = nn.GRU(input_size=100, hidden_size=256, batch_first=True) # no cell state
```

LSTMs/GRUs are largely superseded by Transformers for long sequences (parallelizable, better long-range dependencies), but remain useful for streaming/low-latency or small-data settings. See Part V.

14 Custom Transforms & torchvision

```
from torchvision import transforms

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]), # ImageNet stats
])
dataset = torchvision.datasets.ImageFolder("data/", transform=transform)
```

15 Evaluation & Metrics

```
model.eval()
correct, total = 0, 0
with torch.no_grad():
    for x, y in test_loader:
        preds = model(x).argmax(dim=1)
        correct += (preds == y).sum().item()
        total += y.size(0)
accuracy = correct / total
```

Metric	Used for
Accuracy / F1 / Precision / Recall	Classification
Confusion matrix	Per-class classification error analysis
MSE / MAE / R^2	Regression
IoU / mAP	Object detection, segmentation
Perplexity / BLEU / ROUGE	Language modeling, generation (see Part VI)

16 Logging: TensorBoard

```
from torch.utils.tensorboard import SummaryWriter

writer = SummaryWriter(log_dir="runs/exp1")
writer.add_scalar("train/loss", loss.item(), global_step=step)
writer.add_scalar("val/accuracy", acc, global_step=epoch)
writer.add_image("sample", img_tensor, global_step=epoch)
writer.close()
# then: tensorboard --logdir runs/
```

Part III PyTorch — Advanced

17 Custom Autograd Functions

Define your own forward/backward pass when you need a non-standard gradient (e.g. a straight-through estimator, a custom CUDA kernel, or gradient reversal).

```
class GradientReversal(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, lambd):
        ctx.lambd = lambd
        return x.view_as(x)

    @staticmethod
    def backward(ctx, grad_output):
        return -ctx.lambd * grad_output, None # one grad per forward input

grad_reverse = GradientReversal.apply
y = grad_reverse(x, 1.0)
```

18 Hooks

```
# Forward hook: inspect/modify activations
def fwd_hook(module, input, output):
    print(module.__class__.__name__, output.shape)

handle = model.fc1.register_forward_hook(fwd_hook)
handle.remove()  # always remove when done

# Backward hook: inspect/modify gradients flowing through a tensor
def grad_hook(grad):
    return grad.clamp(-1, 1)  # e.g. manual gradient clipping on one tensor

x.register_hook(grad_hook)

# Full-module backward hook (captures grad w.r.t. inputs/outputs of a module)
model.fc1.register_full_backward_hook(lambda m, gin, gout: None)
```

Hooks are the standard tool for feature extraction (e.g. pulling intermediate CNN activations for style transfer / Grad-CAM) and for debugging exploding/vanishing gradients layer-by-layer.

19 Mixed Precision Training (AMP)

Runs eligible ops in float16/bfloat16 for speed and memory savings while keeping a master copy of weights and loss scaling in float32 for stability.

```
scaler = torch.cuda.amp.GradScaler()

for x, y in loader:
    x, y = x.to(device), y.to(device)
    optimizer.zero_grad()

    with torch.autocast(device_type="cuda", dtype=torch.float16):
        logits = model(x)
        loss = criterion(logits, y)

    scaler.scale(loss).backward()  # scale loss to avoid float16 underflow
    scaler.step(optimizer)         # unscales grads, then optimizer.step()
    scaler.update()                # adjust scale factor for next iteration
```

On Ampere+ GPUs, bfloat16 needs no loss scaling (same exponent range as float32) – simpler: just wrap the forward pass in autocast(dtype=torch.bfloat16) and skip GradScaler.

20 Distributed Training (DDP)

DistributedDataParallel runs one process per GPU, each with a full model replica; gradients are averaged across processes via all-reduce after each backward pass.

```
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

dist.init_process_group(backend="nccl")
torch.cuda.set_device(local_rank)
model = model.to(local_rank)
model = DDP(model, device_ids=[local_rank])

sampler = torch.utils.data.distributed.DistributedSampler(dataset)
loader = DataLoader(dataset, batch_size=32, sampler=sampler)

# launch with: torchrun --nproc_per_node=4 train.py
```

Strategy	When
DataParallel	Quick single-node hack; slower, single process, avoid for real training
DistributedDataParallel	Standard for multi-GPU/multi-node; near-linear scaling
FSDP (Fully Sharded DP)	Shards optimizer states/gradients/parameters across GPUs – needed for very large models (LLM pretraining/fine-tuning) that don't fit on one GPU
Pipeline / Tensor Parallel	Splits a single model's layers or matrices across devices – used for extreme-scale LLMs (paired with libraries like Megatron/DeepSpeed)

21 Compilation & Export

```
# torch.compile - JIT-compiles the model graph for speed (PyTorch 2.x)
model = torch.compile(model) # drop-in; often 1.3-2x speedup

# TorchScript - serialize a model independent of Python (for C++/mobile deployment)
scripted = torch.jit.script(model)
scripted.save("model.pt")
loaded = torch.jit.load("model.pt")

traced = torch.jit.trace(model, example_input) # records ops for a fixed input shape

# ONNX export - interoperability with other runtimes (TensorRT, ONNXRuntime)
torch.onnx.export(model, example_input, "model.onnx", opset_version=17)
```

22 Quantization

Reduces weight/activation precision (e.g. float32 → int8) to shrink model size and speed up inference, at a small accuracy cost.

```
# Dynamic quantization - weights quantized ahead of time, activations at runtime
quantized = torch.quantization.quantize_dynamic(
    model, {nn.Linear}, dtype=torch.qint8
)

# Static quantization - both weights & activations pre-quantized (needs calibration data)
model.qconfig = torch.quantization.get_default_qconfig("fbgemm")
torch.quantization.prepare(model, inplace=True)
# ... run calibration batches through model ...
torch.quantization.convert(model, inplace=True)
```

For LLMs specifically, see the dedicated quantization methods (GPTQ, AWQ, bitsandbytes INT8/NF4) in §50.

23 Profiling & Performance Debugging

```
from torch.profiler import profile, ProfilerActivity

with profile(activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
             record_shapes=True) as prof:
    for _ in range(5):
        output = model(input)
        output.sum().backward()

print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=10))
prof.export_chrome_trace("trace.json") # view in chrome://tracing
```

Symptom	Likely cause / fix
GPU util low, CPU high	Data loading bottleneck – increase <code>num_workers</code> , use <code>pin_memory</code>
OOM (out of memory)	Reduce batch size, use AMP, gradient checkpointing, or FSDP/DeepSpeed
Loss is NaN	Lower LR, add gradient clipping, check for div-by-zero / log(0)
Training slower after <code>torch.compile</code>	First iteration compiles (warmup) – measure after a few steps

24 Gradient Checkpointing (Activation Recomputation)

Trades compute for memory: activations are discarded after the forward pass and recomputed during backward, instead of all being kept in memory. Essential for training very deep nets / long-context Transformers on limited VRAM.

```
from torch.utils.checkpoint import checkpoint

def forward(self, x):
    x = checkpoint(self.block1, x, use_reentrant=False)
    x = checkpoint(self.block2, x, use_reentrant=False)
    return x
```

Part IV Deep Learning — Foundations

25 The Neuron & Backpropagation

A single unit computes $z = w^\top x + b$, then applies a nonlinearity $a = \phi(z)$. Stacking layers gives a composition of functions; **backpropagation** is just the chain rule applied efficiently, layer by layer, from the loss back to the inputs.

Chain rule for a layer

$$\frac{\partial \mathcal{L}}{\partial w^{(l)}} = \frac{\partial \mathcal{L}}{\partial a^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial w^{(l)}} = \delta^{(l)} \cdot x^{(l-1)\top}$$

where $\delta^{(l)} = (W^{(l+1)\top} \delta^{(l+1)}) \odot \phi'(z^{(l)})$ is the error signal propagated backward.

26 Weight Initialization

Poor initialization causes activations/gradients to vanish or explode as depth increases.

Scheme	PyTorch	Best for
Xavier/Glorot	<code>nn.init.xavier_uniform_</code>	Tanh / Sigmoid activations
He/Kaiming	<code>nn.init.kaiming_normal_</code>	ReLU / GELU family (default for CNNs)
Zeros/Ones	<code>nn.init.zeros_/ones_</code>	Biases; final layer of residual/gating branches
Orthogonal	<code>nn.init.orthogonal_</code>	RNNs (preserves gradient norm through time)

```
def init_weights(m):
    if isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight, nonlinearity='relu')
        nn.init.zeros_(m.bias)

model.apply(init_weights)
```

27 Normalization Techniques

General normalization form

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

Variants differ only in *which dimensions* μ, σ^2 are computed over.

Type	Statistics computed over
BatchNorm	Per-channel, across the batch + spatial dims – depends on batch size
LayerNorm	Per-sample, across the feature dim – batch-size independent (Transformers)
InstanceNorm	Per-sample, per-channel, across spatial dims – style transfer
GroupNorm	Per-sample, over a group of channels – good for small batch sizes
RMSNorm	Like LayerNorm but skips mean-centering, only rescales by RMS – faster, used in LLaMA

28 Optimization Algorithms

SGD with Momentum

$$v_t = \mu v_{t-1} + \nabla \mathcal{L}(\theta_{t-1}), \quad \theta_t = \theta_{t-1} - \eta v_t$$

Adam

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

m_t (1st moment) tracks the mean gradient direction; v_t (2nd moment) adapts the per-parameter step size.

29 Regularization & Generalization

Technique	Idea
L2 / weight decay	Penalize large weights $\rightarrow \mathcal{L} + \lambda \ \theta\ _2^2$, smoother decision boundary
L1	Penalize $\ \theta\ _1$, encourages sparsity (many weights exactly 0)
Dropout	Randomly disable units – approximates ensembling many sub-networks
Label smoothing	Soften one-hot targets (e.g. 0.9/0.1 instead of 1/0) – reduces overconfidence
Mixup / CutMix	Train on convex combinations of pairs of examples/labels
Early stopping	Stop when validation loss stops improving
Data augmentation	Synthetic label-preserving input variations

Bias–Variance Tradeoff

$$\mathbb{E}[(\hat{f}(x) - f(x))^2] = \underbrace{(\text{Bias})^2}_{\text{underfitting}} + \underbrace{\text{Variance}}_{\text{overfitting}} + \text{Irreducible Error}$$

30 Vanishing & Exploding Gradients

Deep networks multiply many Jacobians during backprop; if their singular values are consistently < 1 or > 1 , gradients shrink or blow up exponentially with depth.

Mitigation	Mechanism
Residual/skip connections	Provide a direct gradient path (identity shortcut), used in ResNets & Transformers
Normalization (LN/BN)	Keeps activation scale stable across layers
Better activations (ReLU/GELU)	Avoid the near-zero-gradient saturation regions of sigmoid/tanh
Gradient clipping	Caps gradient norm to a max value each step
Careful initialization	See §26

31 CNN Architecture Evolution (Quick Reference)

Model	Key idea
LeNet-5 (1998)	First practical CNN; conv + pool + FC for digit recognition
AlexNet (2012)	Deeper, ReLU, dropout, GPU training – kicked off the deep learning boom
VGG (2014)	Very uniform 3x3 conv stacks; simplicity, depth
GoogLeNet/Inception	Parallel multi-scale conv branches per block
ResNet (2015)	Residual/skip connections – enabled 100+ layer networks
EfficientNet	Compound scaling of depth/width/resolution together
Vision Transformer (ViT)	Treats image patches as a token sequence, uses a Transformer encoder (§39)

32 Hyperparameter Tuning Checklist

- Start with a known-good LR for the optimizer (Adam: $1\text{e-}3$, AdamW+Transformer: $1\text{e-}4$ – $5\text{e-}4$).

- Tune LR and batch size together – larger batch often wants a (roughly linearly) larger LR.
- Use a small subset of data to first confirm the model can *overfit* it (sanity check for bugs).
- Log everything: loss curves, gradient norms, LR schedule.
- Prefer random search or Bayesian optimization (e.g. Optuna) over grid search in high dimensions.

Part V Transformers

33 Why Transformers? (Motivation)

RNNs process sequences step-by-step: this is inherently sequential (slow to train, hard to parallelize) and struggles with long-range dependencies even with LSTM/GRU gating. **Transformers** (Vaswani et al., 2017, "Attention Is All You Need") replace recurrence entirely with **self-attention**, letting every position attend directly to every other position in parallel.

	RNN / LSTM	Transformer
Parallelization	Sequential (slow)	Fully parallel over sequence positions
Long-range deps.	Degrades with distance	Direct, constant path length
Compute per layer	$O(n)$	$O(n^2)$ in sequence length (attention matrix)
Position info	Implicit (order of processing)	Must be added explicitly (positional encoding)

34 Self-Attention

Each token produces a *Query*, *Key*, and *Value* vector via learned linear projections. A token's new representation is a weighted sum of all Value vectors, where weights come from Query–Key similarity.

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

$Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{n \times d_k}$, $V \in \mathbb{R}^{n \times d_v}$. Dividing by $\sqrt{d_k}$ prevents dot products from growing large and pushing softmax into a saturated (near one-hot, tiny-gradient) regime.

```
import torch, torch.nn.functional as F

def self_attention(x, Wq, Wk, Wv):
    Q, K, V = x @ Wq, x @ Wk, x @ Wv          # [seq, d_k], [seq, d_k], [seq, d_v]
    scores = Q @ K.transpose(-2, -1) / (K.size(-1) ** 0.5)  # [seq, seq]
    weights = F.softmax(scores, dim=-1)
    return weights @ V                          # [seq, d_v]
```

Causal (masked) self-attention, used for autoregressive LLMs, adds $-\infty$ to `scores` above the diagonal before softmax so a token cannot attend to future positions.

35 Multi-Head Attention

Instead of one attention function, project into h smaller subspaces ("heads") in parallel, each free to learn a different kind of relationship (e.g. syntax vs. coreference), then concatenate and mix.

Multi-Head Attention

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \quad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

```
mha = nn.MultiheadAttention(embed_dim=512, num_heads=8, batch_first=True)
x = torch.randn(32, 100, 512)          # [batch, seq_len, embed_dim]
out, attn_weights = mha(query=x, key=x, value=x)  # self-attention
# out: [batch, seq_len, embed_dim]
```



```
causal_mask = nn.Transformer.generate_square_subsequent_mask(100)
out, _ = mha(x, x, x, attn_mask=causal_mask) # causal (decoder) attention
```

36 Positional Encoding

Attention itself is permutation-invariant (no notion of order), so position information must be injected.

Scheme	Idea
Sinusoidal (original)	Fixed sin / cos functions of position at different frequencies, added to embeddings
Learned absolute	A trainable embedding table indexed by position (BERT, GPT-2)
RoPE (Rotary)	Rotates Q/K vectors by an angle proportional to position – encodes <i>relative</i> position naturally; used in LLaMA, GPT-NeoX
ALiBi	Adds a distance-proportional bias directly to attention scores instead of modifying embeddings – extrapolates well to longer sequences

Sinusoidal Positional Encoding

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

37 The Transformer Block

Each block = Multi-Head Attention + a position-wise Feed-Forward Network (FFN), each wrapped in a residual connection and normalization.

Pre-LN Transformer Block (modern default)

$$x = x + \text{MHA}(\text{LN}(x)), \quad x = x + \text{FFN}(\text{LN}(x))$$

Pre-LN (normalize *before* the sublayer) trains more stably at depth than the original Post-LN; almost all modern LLMs use Pre-LN.

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model=512, n_heads=8, d_ff=2048, dropout=0.1):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = nn.MultiheadAttention(d_model, n_heads, dropout=dropout, batch_first=True)
        self.ln2 = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff), nn.GELU(), nn.Linear(d_ff, d_model)
        )
        self.drop = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        h = self.ln1(x)
        attn_out, _ = self.attn(h, h, h, attn_mask=mask)
        x = x + self.drop(attn_out)
        x = x + self.drop(self.ffn(self.ln2(x)))
        return x
```

38 Encoder–Decoder Architecture

Architecture	Attention pattern & use case
Encoder-only	Bidirectional self-attention; good for understanding tasks (classification, embeddings) – e.g. BERT
Decoder-only	Causal (masked) self-attention; autoregressive generation – e.g. GPT, LLaMA (most modern LLMs)
Encoder–Decoder	Bidirectional encoder + causal decoder with <i>cross-attention</i> to encoder outputs – e.g. T5, original Transformer, translation

```
transformer = nn.Transformer(d_model=512, nhead=8,
                             num_encoder_layers=6, num_decoder_layers=6,
```

```

batch_first=True)
out = transformer(src, tgt, tgt_mask=causal_mask)  # src: encoder input, tgt: decoder input

```

39 Vision Transformer (ViT)

Splits an image into fixed-size patches (e.g. 16×16), linearly projects each into an embedding (a "patch token"), prepends a learnable [CLS] token, adds positional embeddings, and feeds the sequence through a standard Transformer encoder.

```

patches = img.unfold(2, 16, 16).unfold(3, 16, 16)  # break image into 16x16 patches
patches = patches.reshape(B, C, -1, 16, 16).permute(0, 2, 1, 3, 4).flatten(2)
tokens = patch_embed(patches)  # nn.Linear(16*16*C, d_model)
tokens = torch.cat([cls_token.expand(B, -1, -1), tokens], dim=1) + pos_embed

```

40 Efficient & Long-Context Attention

Method	Idea
FlashAttention	IO-aware fused kernel – computes exact attention without materializing the full $n \times n$ score matrix in slow memory; large speed/memory win, no accuracy tradeoff
Sparse attention	Each token attends to only a subset of positions (local window + a few global tokens)
Linear attention	Approximates attention as a kernel feature map, reducing complexity from $O(n^2)$ to $O(n)$
Sliding-window attention	Fixed local window per token (e.g. Mistral) – constant memory per step
Multi-Query / Grouped-Query Attn.	Share Key/Value projections across (groups of) heads – shrinks the KV-cache for faster inference (§49)

Part VI Large Language Models (LLMs)

41 Tokenization

LLMs don't operate on raw characters or whole words directly – text is split into **subword tokens**, balancing vocabulary size against sequence length.

Algorithm	Idea
BPE (Byte-Pair Encoding)	Iteratively merges the most frequent adjacent symbol pair into a new token – used by GPT-2/GPT-3/LLaMA
WordPiece	Similar to BPE but merges based on likelihood gain – used by BERT
SentencePiece	Language-agnostic, treats text as a raw stream (handles whitespace as a symbol) – used by T5, LLaMA
Byte-level BPE	Operates on raw UTF-8 bytes – guarantees no out-of-vocabulary tokens (GPT-2/GPT-3/GPT-4)

```

from transformers import AutoTokenizer

tok = AutoTokenizer.from_pretrained("gpt2")
ids = tok.encode("Transformers are powerful!")  # -> list of integer token IDs
tok.decode(ids)  # back to text
tok("Hello world", return_tensors="pt", padding=True, truncation=True)

```

Rule of thumb: 1 token $\approx$ 0.75 English words. A model's *context window* is measured in tokens, not characters or words.

42 Embeddings & Input Representation

Each token ID indexes into a learned embedding table (`nn.Embedding`) to get a dense vector; positional information (§36) is then added or fused in.

```
tok_embed = nn.Embedding(vocab_size, d_model)
x = tok_embed(input_ids) * (d_model ** 0.5)  # scale, as in the original Transformer
```

43 Pretraining Objectives

Objective	Description
Causal LM (CLM)	Predict the next token given all previous tokens; trained with a causal mask – GPT, LLaMA family
Masked LM (MLM)	Randomly mask ~15% of tokens, predict them from bidirectional context – BERT
Span corruption	Mask contiguous spans, predict the missing span as a sequence – T5
Prefix LM	Bidirectional over a prefix, causal over the continuation – UL2, PaLM variants

Causal LM loss

$$\mathcal{L} = - \sum_{t=1}^n \log P(x_t \mid x_{<t}; \theta)$$

This is exactly `nn.CrossEntropyLoss` applied at every sequence position, averaged.

44 Model Family Tree

Model	Type	Notable trait
BERT	Encoder-only	Bidirectional; strong for classification/embeddings, not generation
GPT-2/3/4	Decoder-only	Autoregressive causal LM; the dominant modern paradigm
T5 / Flan-T5	Encoder-Decoder	Frames every task as text-to-text
LLaMA / Llama 2/3	Decoder-only	RMSNorm, RoPE, SwiGLU activations; open-weight family
Mistral / Mixtral	Decoder-only (dense / MoE)	Sliding-window attention; Mixtral uses Mixture-of-Experts
PaLM / Gemini	Decoder-only	Very large scale, multimodal (Gemini)

45 Fine-Tuning: Full vs. Parameter-Efficient (PEFT)

Approach	Notes
Full fine-tuning	Updates every parameter; best quality ceiling, but needs the most memory/compute and a full checkpoint per task
LoRA	Freezes base weights, learns small low-rank update matrices (§46) – cheap, swappable adapters
QLoRA	LoRA on top of a 4-bit quantized frozen base model – fine-tune large models on a single consumer GPU
Prompt / Prefix tuning	Learns a small set of continuous "soft prompt" vectors, base model fully frozen
Adapters	Small bottleneck feed-forward modules inserted between frozen layers

46 LoRA (Low-Rank Adaptation)

LoRA update

$$W' = W_0 + \Delta W = W_0 + BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k)$$

W_0 (the pretrained weight) stays *frozen*; only A and B are trained, cutting trainable parameters by orders of magnitude.

```
from peft import LoraConfig, get_peft_model

config = LoraConfig(
```

```

r=8, lora_alpha=16, lora_dropout=0.05,
target_modules=["q_proj", "v_proj"],      # which weight matrices to adapt
task_type="CAUSAL_LM",
)
model = get_peft_model(base_model, config)
model.print_trainable_parameters()      # e.g. "0.06% trainable"

```

47 Instruction Tuning, RLHF & DPO

Stage	Purpose
Supervised Fine-Tuning (SFT)	Fine-tune the base LM on (instruction, ideal response) pairs
Reward Modeling (RM)	Train a model to score responses, from human preference comparisons (A vs B)
RLHF (PPO)	Use the reward model as the RL signal to further optimize the LM (see Part VII for PPO mechanics)
DPO (Direct Preference Optimization)	Skips the separate reward model + RL loop – directly optimizes the policy on preference pairs with a classification-style loss

RLHF/DPO align a model to human *preferences* (helpful, harmless, honest) – they don't add new knowledge; that comes from pretraining.

48 Decoding / Generation Strategies

Strategy	Behavior
Greedy	Always pick the highest-probability token – deterministic, often repetitive
Beam search	Track the top- k partial sequences by cumulative probability – common for translation/-summarization
Temperature	Rescale logits by $1/T$ before softmax; $T < 1$ sharpens, $T > 1$ flattens the distribution
Top-k sampling	Sample only from the k most likely tokens
Top-p (nucleus) sampling	Sample from the smallest set of tokens whose cumulative probability exceeds p
Repetition penalty	Downweight tokens already generated, to reduce loops

```

from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("gpt2")
out = model.generate(
    input_ids, max_new_tokens=100,
    do_sample=True, temperature=0.8, top_p=0.9, top_k=50,
    repetition_penalty=1.2,
)

```

49 KV-Cache & Inference Optimization

During autoregressive generation, recomputing Key/Value projections for *all* previous tokens at every new step is wasteful. The **KV-cache** stores past Keys/Values and reuses them, so each new step only computes attention for the newest token.

Technique	Effect
KV-cache	$O(n)$ instead of $O(n^2)$ total work across a full generation
Multi-/Grouped-Query Attention	Shrinks KV-cache size by sharing K/V across heads (§40)
PagedAttention (vLLM)	Manages the KV-cache like OS virtual memory pages – reduces fragmentation, enables higher batch throughput
Speculative decoding	A small "draft" model proposes several tokens; the large model verifies them in one pass – reduces effective latency
Continuous batching	Dynamically add/remove sequences from a running batch instead of static batching

50 Quantization for LLMs

Method	Notes
INT8 (bitsandbytes)	Simple post-training quantization, minor quality loss
NF4 / 4-bit (QLoRA)	4-bit NormalFloat, tuned for the distribution of neural net weights; enables QLoRA fine-tuning
GPTQ	Post-training quantization using layer-wise second-order (Hessian) error correction
AWQ (Activation-aware)	Preserves the small subset of weights most important for activations, quantizes the rest aggressively

```
from transformers import BitsAndBytesConfig

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
)
model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-7b-hf",
                                             quantization_config=bnb_config)
```

51 Extending Context Length

- **Position interpolation:** rescale RoPE position indices to compress a longer sequence into the range the model was trained on.
- **NTK-aware / YaRN scaling:** adjust RoPE's base frequency non-uniformly across dimensions for less quality loss.
- **ALiBi:** extrapolates naturally since it's a fixed linear bias, not a learned/rotated embedding.
- **Sliding-window attention:** bounds compute per token regardless of total context length (§40).

52 Prompt Engineering Quick Reference

Technique	Use
Zero-shot	Ask directly, no examples – fastest, works for simple/common tasks
Few-shot	Provide 2–5 input/output examples in-context before the real query
Chain-of-thought	Ask the model to reason step-by-step before the final answer – improves multi-step reasoning
System prompt	Persistent instruction setting role, tone, and constraints for the whole conversation
Structured output	Explicitly request JSON/XML with a described schema for reliable parsing
Retrieval-augmented generation (RAG)	Retrieve relevant documents and insert them into the prompt as grounding context

53 Evaluation Metrics

Metric	Measures
Perplexity	exp(average negative log-likelihood per token) – lower is better; intrinsic LM quality
BLEU	N-gram precision overlap with reference text – translation
ROUGE	N-gram recall overlap with reference – summarization
Exact Match / F1	Span overlap with a gold answer – extractive QA
Human / LLM-as-judge preference	Pairwise comparison of response quality – used for RLHF reward modeling and leaderboard evals

54 Hugging Face transformers: Quick Start

```
from transformers import AutoTokenizer, AutoModelForCausalLM, TrainingArguments, Trainer

tok = AutoTokenizer.from_pretrained("gpt2")
model = AutoModelForCausalLM.from_pretrained("gpt2")
```

```
args = TrainingArguments(
    output_dir="out", per_device_train_batch_size=8,
    num_train_epochs=3, learning_rate=5e-5, fp16=True,
    logging_steps=50, save_strategy="epoch",
)
trainer = Trainer(model=model, args=args, train_dataset=train_ds, eval_dataset=eval_ds)
trainer.train()
```

55 Scaling Laws (Brief)

Empirically, LM loss falls as a power law with model size N , dataset size D , and compute C , roughly independently, up to a point where one factor bottlenecks the others.

Chinchilla-style compute-optimal scaling

For a fixed compute budget, loss is minimized by scaling model parameters N and training tokens D in roughly equal proportion ($D \approx 20N$) rather than making the model much bigger than its training data can support.

Part VII Reinforcement Learning

56 Markov Decision Processes (MDPs)

Reinforcement learning frames a problem as an agent interacting with an environment over time: at each step t the agent observes state s_t , takes action a_t , receives reward r_t , and transitions to s_{t+1} .

Symbol	Meaning
$s \in \mathcal{S}$	State
$a \in \mathcal{A}$	Action
$r(s, a)$	Reward function
$P(s' s, a)$	Transition (dynamics) function
$\pi(a s)$	Policy – the agent’s action-selection strategy
$\gamma \in [0, 1]$	Discount factor – trades off immediate vs. future reward
$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$	Return – total discounted future reward from time t

Value & Action-Value Functions

$$V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s], \quad Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$$

V^π = expected return from a state under policy π . Q^π = expected return from a state *having just taken* a specific action.

57 Bellman Equations

Bellman Expectation Equation

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P} [\mathbb{E}_{a' \sim \pi} [Q^\pi(s', a')]]$$

Bellman Optimality Equation

$$Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P} [\max_{a'} Q^*(s', a')]$$

This recursive identity is the basis of value-based RL: if we knew Q^* , the optimal policy is simply $\pi^*(s) = \arg \max_a Q^*(s, a)$.

58 Value-Based Methods: Q-Learning & DQN

Q-learning learns Q^* directly from experience via temporal-difference (TD) updates, without needing a model of the environment.

Tabular Q-learning update

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

DQN (Deep Q-Network) replaces the table with a neural network $Q_\theta(s, a)$ for large/continuous state spaces, stabilized by two key tricks: a **replay buffer** (breaks correlation between consecutive samples) and a **target network** (a slowly-updated copy of Q_θ that stabilizes the TD target).

```
import random, collections
import torch, torch.nn as nn

class QNet(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128), nn.ReLU(),
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, n_actions),
        )
    def forward(self, x):
        return self.net(x)

q_net, target_net = QNet(4, 2), QNet(4, 2)
target_net.load_state_dict(q_net.state_dict())
optimizer = torch.optim.Adam(q_net.parameters(), lr=1e-3)
buffer = collections.deque(maxlen=100_000)

def train_step(batch_size=64, gamma=0.99):
    batch = random.sample(buffer, batch_size)
    s, a, r, s2, done = map(torch.stack, zip(*batch))

    q_values = q_net(s).gather(1, a.unsqueeze(1)).squeeze(1)
    with torch.no_grad():
        target = r + gamma * target_net(s2).max(dim=1)[0] * (1 - done)

    loss = nn.functional.mse_loss(q_values, target)
    optimizer.zero_grad(); loss.backward(); optimizer.step()

# periodically: target_net.load_state_dict(q_net.state_dict())
```

59 Exploration Strategies

Strategy	Idea
ϵ -greedy	With probability ϵ take a random action, else the greedy (best- Q) action; decay ϵ over training
Boltzmann/softmax	Sample actions proportional to $\exp(Q(s, a)/T)$
Entropy bonus	Add $-\beta \mathcal{H}(\pi(\cdot s))$ to the objective so the policy stays stochastic longer (used in A2C/PPO)
Intrinsic motivation	Add a curiosity/novelty bonus reward for visiting unfamiliar states

60 Policy Gradient Methods: REINFORCE

Instead of learning values and deriving a policy, **policy gradient** methods directly parameterize and optimize $\pi_\theta(a | s)$ by gradient ascent on expected return.

Policy Gradient Theorem

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t | s_t) G_t]$$

```
policy = nn.Sequential(nn.Linear(state_dim, 128), nn.ReLU(),
                      nn.Linear(128, n_actions), nn.Softmax(dim=-1))
optimizer = torch.optim.Adam(policy.parameters(), lr=1e-3)

log_probs, rewards = [], []
for t in range(T):
```

```

probs = policy(state)
dist = torch.distributions.Categorical(probs)
action = dist.sample()
log_probs.append(dist.log_prob(action))
state, reward, done, _ = env.step(action.item())
rewards.append(reward)

returns = compute_discounted_returns(rewards, gamma=0.99) # G_t for every t
returns = (returns - returns.mean()) / (returns.std() + 1e-8) # baseline/normalize

loss = -torch.stack([lp * G for lp, G in zip(log_probs, returns)]).sum()
optimizer.zero_grad(); loss.backward(); optimizer.step()

```

Subtracting a **baseline** (e.g. the mean return, or a learned value function $V(s)$) from G_t doesn't bias the gradient but greatly reduces its variance – this is exactly the idea behind Actor-Critic.

61 Actor-Critic & A2C

Two networks: an **actor** $\pi_\theta(a|s)$ that selects actions, and a **critic** $V_\phi(s)$ that estimates state value, used to compute a low-variance **advantage** signal.

Advantage

$$A(s_t, a_t) = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

Actor loss: $-\log \pi_\theta(a_t|s_t) \cdot A(s_t, a_t)$. Critic loss: $(V_\phi(s_t) - (r_t + \gamma V_\phi(s_{t+1})))^2$.

62 PPO (Proximal Policy Optimization)

The most widely used policy-gradient algorithm in practice (including RLHF for LLMs, §47). PPO clips the policy update so a single step can't move the policy too far from the one that collected the data – much more stable than vanilla policy gradient.

PPO Clipped Objective

$$L(\theta) = \mathbb{E}_t \left[\min(\rho_t(\theta) A_t, \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon) A_t) \right], \quad \rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

```

ratio = torch.exp(new_log_probs - old_log_probs.detach())
surrr1 = ratio * advantages
surrr2 = torch.clamp(ratio, 1 - eps, 1 + eps) * advantages
policy_loss = -torch.min(surrr1, surrr2).mean()

loss = policy_loss + vf_coef * value_loss - ent_coef * entropy_bonus
optimizer.zero_grad(); loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=0.5)
optimizer.step()

```

63 RL for LLMs: RLHF Recap

In RLHF (§47), the "environment" is a single-step episode: state = prompt, action = generated response (token by token), reward = the learned reward model's score (often with a KL-divergence penalty against the original SFT model to prevent the policy from drifting too far and "reward hacking"). PPO is applied with the LLM itself as the policy network.

RLHF objective

$$\max_{\theta} \mathbb{E}_{x, y \sim \pi_{\theta}} [R_{\phi}(x, y)] - \beta D_{KL}(\pi_{\theta}(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))$$

64 Quick Reference: Algorithm Families

Family	Examples	On/Off-policy
Value-based	Q-learning, DQN, Double DQN, Dueling DQN	Off-policy
Policy gradient	REINFORCE, A2C/A3C	On-policy
Actor-Critic (advanced)	PPO, TRPO	On-policy
Off-policy Actor-Critic	DDPG, TD3, SAC	Off-policy (continuous control)
Model-based	World Models, MuZero	Either

Part VIII Appendix

65 Common PyTorch Errors & Fixes

Error	Typical cause / fix
<code>RuntimeError: size mismatch / shape errors in matmul</code>	Check tensor shapes with <code>.shape</code> before the op; remember <code>nn.Linear</code> multiplies by W^T
Expected all tensors to be on the same device	Model and data on different devices – move both with <code>.to(device)</code>
CUDA out of memory	Reduce batch size; use AMP (§19); call <code>torch.cuda.empty_cache()</code> ; use gradient checkpointing (§24)
Trying to backward through the graph a second time	You called <code>.backward()</code> twice without <code>retain_graph=True</code> , or reused a tensor across iterations without detaching
Loss is NaN / inf	Lower LR; add gradient clipping; check for <code>log(0)</code> or division by zero; verify input normalization
Element 0 of tensors does not require grad	Calling <code>.backward()</code> on a tensor that was created under <code>torch.no_grad()</code> or via <code>.detach()</code>
Model output doesn't change / loss flat	Optimizer not stepping (missing <code>.step()</code>), LR too small, or gradients never computed (forgot <code>.backward()</code>)
<code>DataLoader worker exited unexpectedly</code>	Often a <code>num_workers</code> / multiprocessing + Jupyter interaction issue – try <code>num_workers=0</code> to isolate
Dropout/BatchNorm giving inconsistent eval results	Forgot <code>model.eval()</code> before validation/inference

66 Shape Debugging Cheatsheet

Data	Conventional shape
Image batch (CNN input)	<code>[batch, channels, height, width]</code> (NCHW)
Sequence batch (<code>batch_first=True</code>)	<code>[batch, seq_len, features]</code>
Transformer hidden states	<code>[batch, seq_len, d_model]</code>
Attention scores	<code>[batch, num_heads, seq_len, seq_len]</code>
Classification logits	<code>[batch, num_classes]</code>
CrossEntropyLoss targets	<code>[batch]</code> , dtype long, values in <code>[0, num_classes)</code>
RNN/LSTM hidden state	<code>[num_layers * num_directions, batch, hidden_size]</code>

```
# Quick shape-tracing pattern while debugging a new model
def trace(name, t):
    print(f"{name}: {tuple(t.shape)} dtype={t.dtype} device={t.device}")

trace("input", x); x = layer1(x); trace("after layer1", x)
```

67 Useful One-Liners

```
sum(p.numel() for p in model.parameters())           # total parameter count
sum(p.numel() for p in model.parameters() if p.requires_grad) # trainable parameter count
torch.manual_seed(42)                                # reproducibility
```

```

next(model.parameters()).device          # which device is the model on?
{n: p.shape for n, p in model.named_parameters()} # inspect every param shape
torch.autograd.set_detect_anomaly(True)    # locate the exact NaN-producing op
del model, optimizer; torch.cuda.empty_cache() # free GPU memory in a notebook
with torch.inference_mode(): ...           # faster than no_grad() for pure inference

```

68 Glossary

Term	Meaning
Epoch	One full pass over the training dataset
Logits	Raw, unnormalized model outputs before softmax/sigmoid
Checkpoint	A saved snapshot of model (and optionally optimizer) state
Context window	Maximum number of tokens an LLM can attend to at once
Fine-tuning	Continuing to train a pretrained model on a new, usually smaller, task-specific dataset
Inference	Running a trained model forward to get predictions (no gradient updates)
Latent space	The internal, learned vector representation space of a model
Overfitting	Model fits training data (incl. its noise) but generalizes poorly to new data
Teacher forcing	Feeding the ground-truth previous token (not the model's own prediction) during sequence training
Zero-shot	Performing a task with no task-specific training examples, only a natural-language description

69 Further Reading

- **PyTorch**: official docs at pytorch.org/docs; the "PyTorch Performance Tuning Guide."
- **Transformers**: Vaswani et al., *"Attention Is All You Need"* (2017); Hugging Face **transformers** documentation.
- **LLMs**: Kaplan et al. and Hoffmann et al. (Chinchilla) on scaling laws; the LLaMA and GPT technical reports.
- **Reinforcement Learning**: Sutton & Barto, *"Reinforcement Learning: An Introduction"*; Schulman et al. on PPO/TRPO.
- **Deep Learning theory**: Goodfellow, Bengio & Courville, *"Deep Learning"* (MIT Press).

This document is a compact reference, not a substitute for the primary papers/docs above – use the section labels (e.g. §46, §62) and the index below to jump straight to a topic.

Index

advantage, [23](#)
Autograd, [5](#)

backpropagation, [13](#)

DistributedDataParallel, [11](#)
DQN, [22](#)

KV-cache, [19](#)

nn.Module, [5](#)

policy gradient, [22](#)

Q-learning, [22](#)

self-attention, [15](#)
subword tokens, [17](#)

Tensor, [4](#)
Transformers, [15](#)